

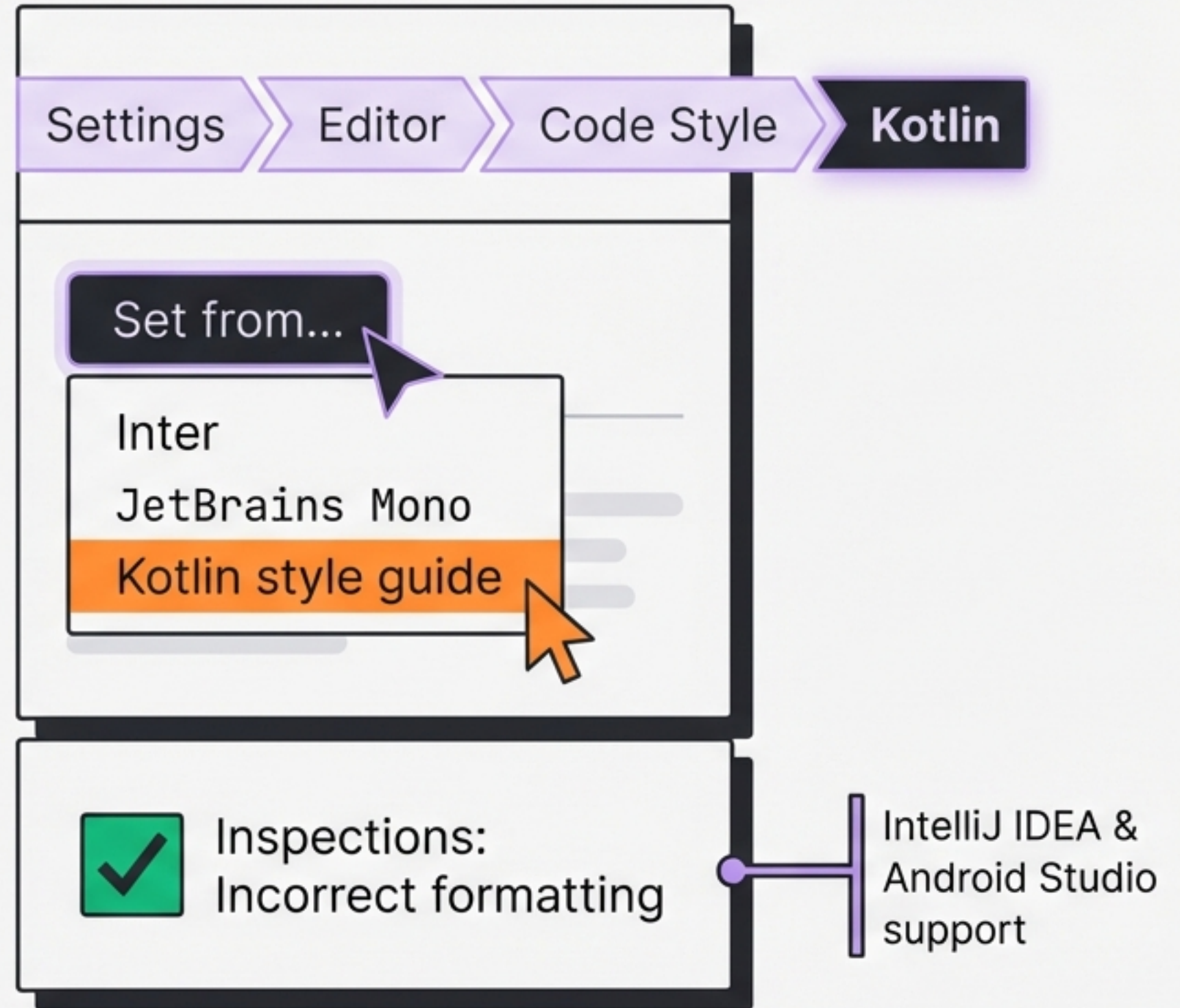


Kotlin Coding Conventions

A Guide to Style, Structure, and Idiomatic Design.

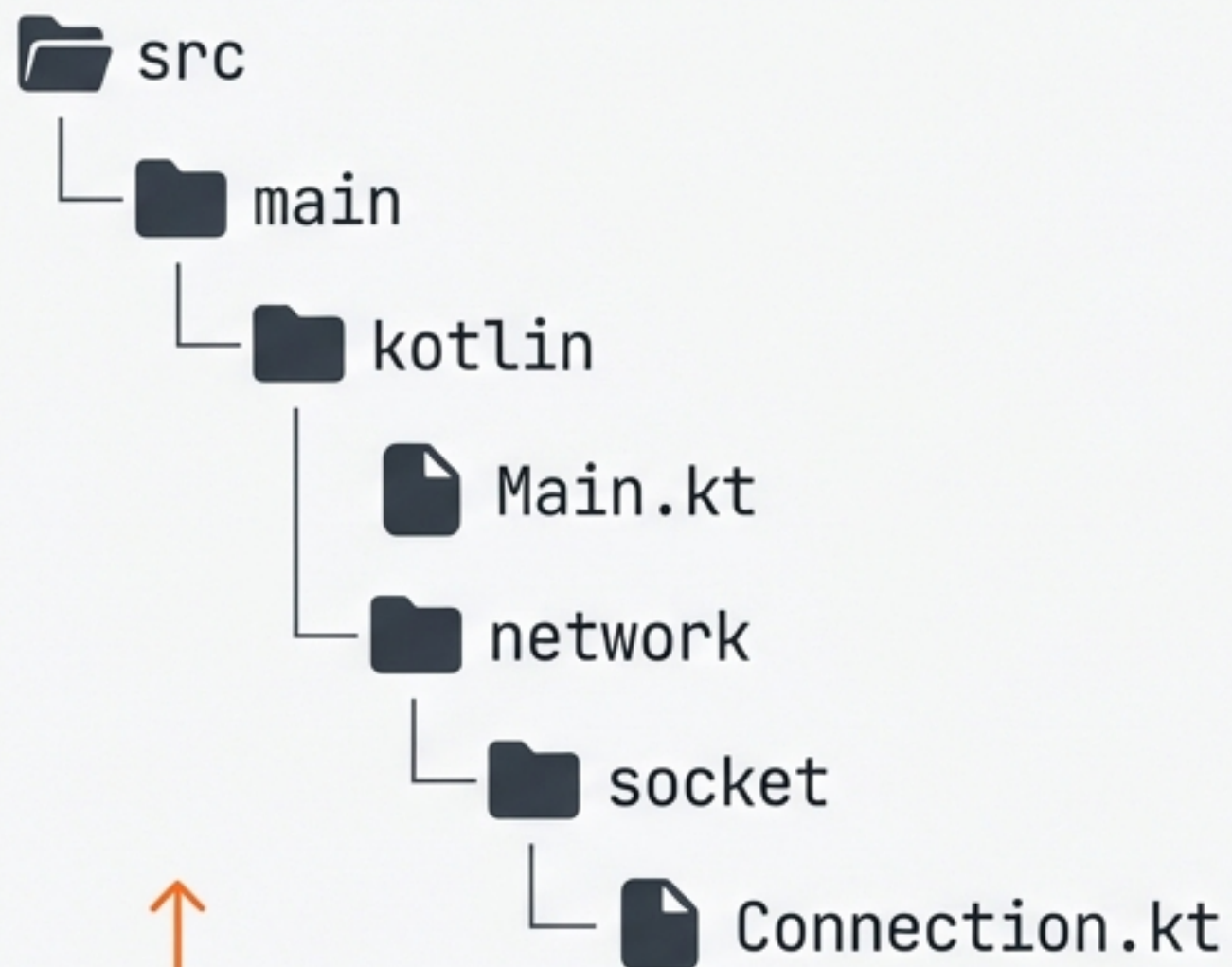
Automate the Rules. Don't Memorize Them.

Conventions are vital, but manual enforcement is inefficient. Configure your IDE to be the enforcer.



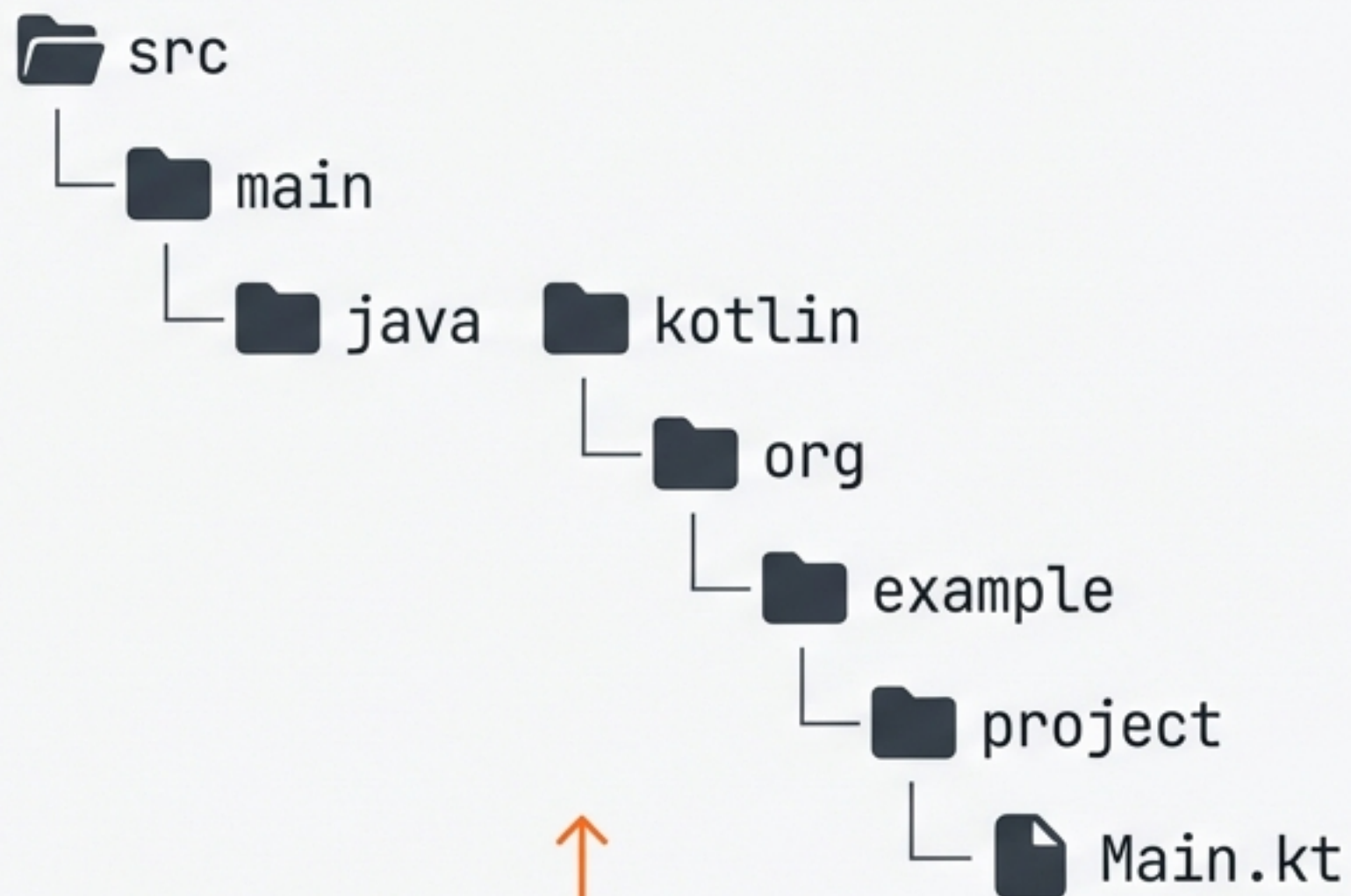
Directory Structure: The Macro View

A. Pure Kotlin Projects



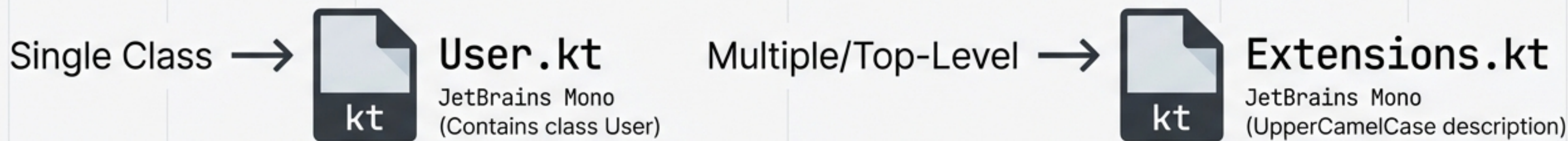
Rule: Omit common root packages (e.g. org.example).

B. Mixed Java/Kotlin (JVM)



Rule: Follow Java package structure exactly.

Source File Names & The Facade Problem



Warning: Multiplatform Collisions



Solution: Use Suffixes



Class Layout & Logical Order

Readability from top to bottom.

```
abstract class           ... {
```

```
        val property: Type = ...  
        
```

```
    constructor(...) { ... }  
    ...  
}
```

```
    fun method(...) { ... }  
    ...  
}
```

```
    companion object {  
        ...  
    }
```

```
}
```

Properties & Initializers
val property: Type = ...

Secondary Constructors
constructor(...) {
 ...
}

Method Declarations
fun method(...) { ... }

Companion Object
companion object { ... }

Don't

- Do not sort alphabetically.
- Do not separate extension methods from the class they extend.
- Do not sort by visibility.

Naming Conventions: The Essentials

Classes & Objects

UpperCamelCase

```
class DeclarationProcessor
```

Functions & Properties

lowerCamelCase

```
fun processDeclarations()
```

Constants

SCREAMING_SNAKE_CASE

```
const val MAX_COUNT = 8
```

Backing Properties

_underscorePrefix

```
private val _elements = ...  
val elements get() = _elements
```


Advanced Naming Nuances

Acronyms

Keep it readable. Capitalize only the first letter if 3+ characters.



XmlFormatter
HttpInputStream



XMLFormatter
HTTPInputStream

(Note: 2 letters like IOStream are uppercase)

Tests

Spaces allowed in backticks (Tests only).

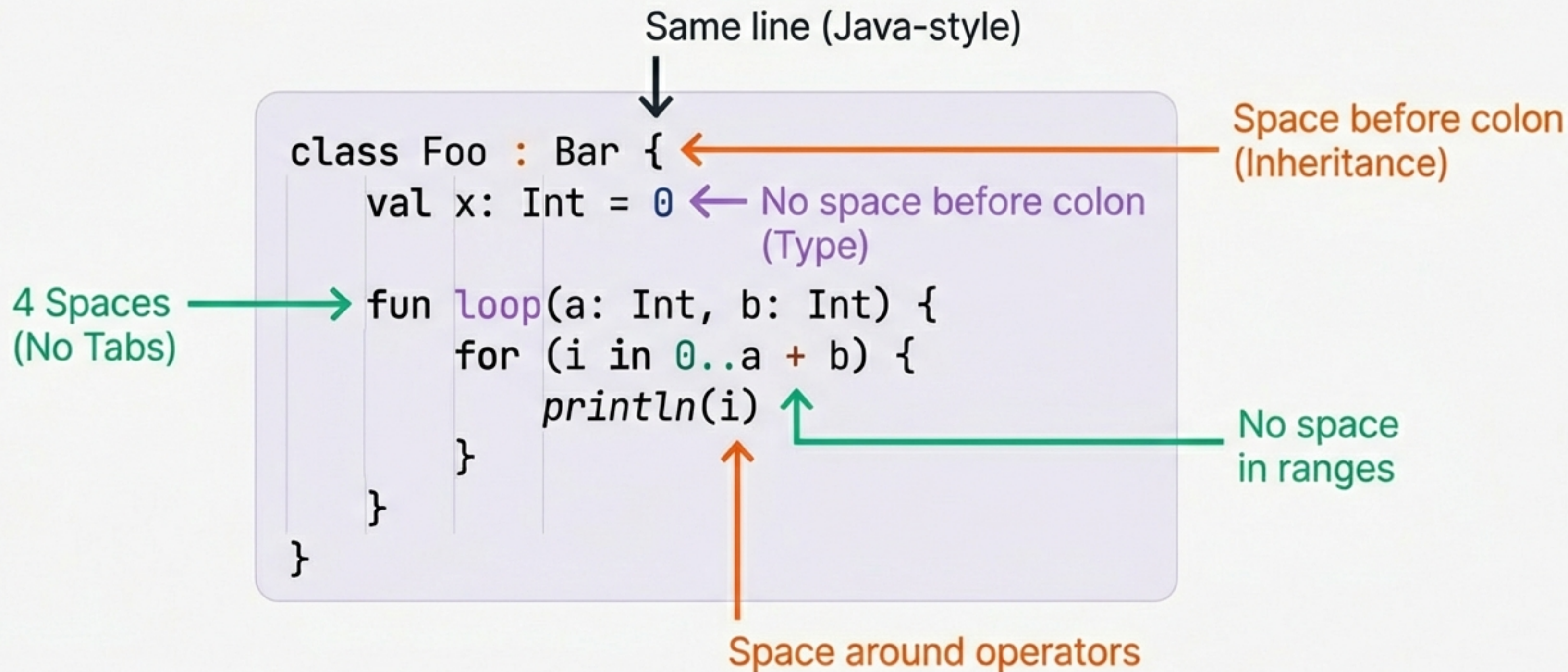
```
fun `ensure everything  
works`() {  
    ...  
}
```

Factory Functions

Can match return type name.

```
fun Foo(): Foo {  
    return FooImpl()  
}
```


Indentation & Horizontal Whitespace

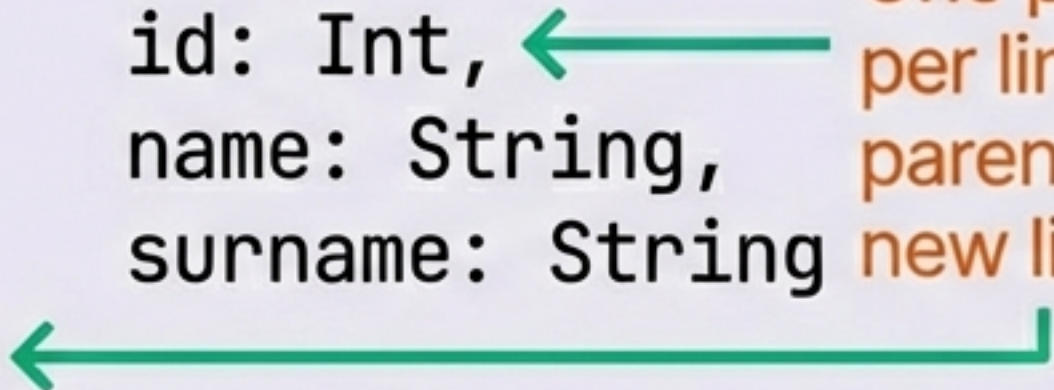


Line Breaks & Wrapping

Function Headers

```
class Person(  
    id: Int,   
    name: String,   
    surname: String  
)  
) : Human(id, name) {
```

One parameter per line. Closing parenthesis on new line.



Chained Calls

```
val anchor = owner  
    ?.firstChild!!  
    .siblings(forward = true)
```



Dot (.) or Safe Call (?.) goes on the next line.

The Version Control Hero: Trailing Commas

```
shift(  
  25,  
  20,  
)
```

Highlight this comma
in bright purple

The Benefits



Cleaner Diffs
(Only changed lines appear).



Easy Reordering
(No syntax errors).



Simplifies Code Gen.

Optional but encouraged.

Documentation (KDoc)

Use standard KDoc syntax.

Link variables directly. Avoid @param tags.

Public members in libraries must be documented.

/**

* Returns the absolute value of the given [\[number\]](#).

*/

fun abs(number: Int): Int { ... }

Explicit return types required for public API stability.

Idiomatic Control Flow

Expressions over Statements.

❌ Statement - Avoid

```
fun getStatus(x: Int): String {  
    if (x == 0) return 'Zero'  
    else return 'NonZero'  
}
```



✅ Expression - Prefer

```
fun getStatus(x: Int) =  
    if (x == 0) 'Zero'  
    else 'NonZero'
```

The Rules

- Binary options: Use 'if'
- 3+ options: Use 'when'
- Loops: Prefer filter/map. Use '0..<n' for open ranges.

Functions, Lambdas, & Arguments

Expression Bodies

```
fun foo() = 1
```



Use = for single
expressions

Lambdas

```
list.filter { it > 10 }
```



Pass outside parenthesis.
Use 'it' for short, non-
nested lambdas.

Named Arguments

```
drawSquare(width = 100,  
            fill = true)
```



Use for clarity, especially
with Booleans.

Immutability & Safety

Variables

`val` > `var`

```
val items = listOf(...)
```



```
var items = arrayListOf(...)
```



Platform Types (Java Interop)

```
// Bad: Implicit platform type  
fun apiCall() = JavaApi.getProp()
```



```
// Good: Explicit type prevents NPEs  
fun apiCall(): String = JavaApi.getProp()
```



Summary: The Kotlin Standard



Automate formatting in IDE settings.



Follow standard directory structures (Pure vs. Mixed).



Prioritize readability: meaningful names and logical class layout.



Use trailing commas and 4-space indentation.



Embrace idioms: Immutability (`val`), Expressions, and KDoc.

Write code for the reader, not just the compiler.